

9) Cliente-Servidor Básico

Um servidor TCP segue um ciclo bem definido de funcionamento. Inicialmente, ele cria um socket, associa esse socket a um endereço IP e uma porta específica utilizando `bind()`, e então entra em modo de escuta com `listen()`. A partir daí, o servidor aguarda conexões de clientes com `accept()`. Quando uma conexão é estabelecida, ocorre a troca de dados por meio de chamadas como `send()` e `recv()`. Após finalizar a comunicação com um cliente, o servidor pode fechar aquela conexão específica, mas continuar ativo para atender novos clientes.

10) Conexões Múltiplas

Para atender múltiplos clientes simultaneamente, o servidor pode adotar diferentes estratégias. Uma abordagem comum é o uso de threads, onde cada cliente é atendido em uma thread separada. Outra opção é o uso de processos, que isolam melhor a execução, porém com maior custo. Também é possível utilizar I/O não bloqueante com mecanismos como `select`, `poll` ou `epoll`, permitindo que um único processo gerencie várias conexões. A escolha depende do equilíbrio entre desempenho, simplicidade e escalabilidade desejada.

11) UDP vs TCP em Chat em Grupo

Ao utilizar UDP em um chat em grupo, o desenvolvedor precisa implementar manualmente mecanismos que o TCP já oferece. Isso inclui garantir a ordem das mensagens, confiabilidade na entrega e controle de perdas. Para isso, podem ser usados números de sequência, confirmações de recebimento (ACKs), retransmissões em caso de perda e buffers para reordenar mensagens. Sem essas estratégias, as mensagens podem chegar fora de ordem, duplicadas ou nem chegar ao destino.

12) Serialização

Para enviar estruturas de dados complexas entre sistemas, é necessário convertê-las para um formato transmissível em bytes. Isso é feito por meio de serialização. Formatos comuns incluem JSON, XML, Protobuf, MessagePack e Avro. JSON é mais simples e legível, porém menos eficiente. Protobuf e Avro são mais compactos e rápidos, sendo ideais para sistemas distribuídos. A escolha do formato impacta diretamente na performance e interoperabilidade do sistema.

13) Heartbeat

O mecanismo de heartbeat é utilizado para verificar se uma conexão ainda está ativa. Ele funciona através do envio periódico de mensagens entre cliente e servidor. Se uma das partes deixa de receber essas mensagens dentro de um intervalo de tempo definido, considera-se que a outra está indisponível. Esse mecanismo é essencial em sistemas distribuídos para detectar falhas rapidamente e tomar decisões, como reconexão ou redistribuição de carga.

14) Balanceamento

O balanceamento de carga é utilizado para distribuir requisições entre múltiplos servidores, evitando sobrecarga em um único nó. Isso pode ser feito através de um balanceador central ou por algoritmos distribuídos. Estratégias comuns incluem round-robin, menor número de conexões e hashing consistente. O objetivo principal é melhorar a escalabilidade e a disponibilidade do sistema, garantindo que os recursos sejam utilizados de forma eficiente.

15) Broadcast

Em redes UDP, o broadcast permite enviar uma mensagem para múltiplos dispositivos simultaneamente. Isso pode ser feito utilizando endereços de broadcast ou mantendo uma lista de destinatários. No entanto, o UDP não garante entrega, ordem ou exclusividade, o que significa que mensagens podem ser perdidas, duplicadas ou recebidas fora de ordem. Além disso, o uso excessivo de broadcast pode gerar tráfego desnecessário na rede.

16) Timeouts

Timeouts são utilizados para evitar que o sistema fique bloqueado indefinidamente esperando uma resposta. Em sockets, é possível definir um tempo máximo de espera para operações de envio e recebimento. Em Python, isso pode ser feito com `settimeout()`. Caso o tempo seja excedido, uma exceção é lançada, permitindo que o sistema trate a situação, como tentar novamente ou encerrar a operação. Isso aumenta a robustez do sistema.

17) Resiliência

Para tornar um servidor TCP resiliente a entradas inválidas ou maliciosas, é fundamental validar todos os dados recebidos, limitar o tamanho das mensagens e tratar exceções adequadamente. O servidor não deve assumir que os dados recebidos são corretos. Além disso, cada conexão deve ser tratada de forma isolada, evitando que um erro em um cliente afete os demais. O uso de logs e mecanismos de rejeição de mensagens inválidas também contribui para a estabilidade do sistema.

18) Camadas

A comunicação em redes é organizada em camadas. A camada de aplicação define o significado dos dados, como requisições de arquivos ou mensagens de chat. Já a camada de transporte, como TCP ou UDP, é responsável por entregar os dados entre os dispositivos. Por exemplo, em uma aplicação de envio de arquivos, a camada de aplicação define o formato da requisição, enquanto o TCP garante que os dados cheguem corretamente e na ordem correta ao destino.